

✓ Airline Customer Satisfaction Prediction

Binomial Logistic Regression Analysis & Business Insights

Project Overview: This notebook builds a end-to-end Logistic Regression classification pipeline to predict airline passenger satisfaction. It handles preprocessing, feature encoding, model training, evaluation (Confusion Matrix, Precision, Recall), and coefficient interpretation to drive actionable business strategies.

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
# =====
# Airline Passenger Satisfaction Analysis
# Binomial Logistic Regression Inference Engine
#
# Description: This script implements the complete data transformation,
# predictive modeling, and business coefficient parsing pipeline.
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, precision_score, recall_score

# Structural Configurations
np.random.seed(42)
sns.set_theme(style="whitegrid")

# -----
# 1. Controlled Data Realization & Target Engineering
# -----

# Mock initialization mapping standard baseline operational limits
n_samples = 1000
mock_data = {
    'Age': np.random.randint(18, 70, size=n_samples),
    'Flight_Distance': np.random.randint(200, 4000, size=n_samples),
    'Inflight_Wifi_Quality': np.random.randint(1, 6, size=n_samples),
    'Seat_Comfort': np.random.randint(1, 6, size=n_samples),
    'On_time_Departure_Delay': np.random.exponential(scale=15, size=n_samples).astype(int),
    'Customer_Type': np.random.choice(['Loyal', 'Disloyal'], size=n_samples, p=[0.8, 0.2]),
    'Class': np.random.choice(['Business', 'Eco', 'Eco Plus'], size=n_samples, p=[0.5, 0.4, 0.1])
    'satisfaction': np.random.choice(['Satisfied', 'Dissatisfied'], size=n_samples)
}
df = pd.DataFrame(mock_data)

# Structural equation matching ground realities
satisfaction_prob = (
    (df['Inflight_Wifi_Quality'] * 0.4) +
    (df['Seat_Comfort'] * 0.3) +
    (df['Class'] == 'Business').astype(int) * 1.5 -
```

```

(df['On_time_Departure_Delay'] * 0.02)
)
prob_normalized = 1 / (1 + np.exp(-satisfaction_prob + 2))
df['satisfaction'] = np.where(prob_normalized > 0.5, 'Satisfied', 'Dissatisfied')

# Target encoding execution
df['satisfaction'] = df['satisfaction'].map({'Satisfied': 1, 'Dissatisfied': 0})
print("=== FIRST 5 ROWS OF DATASET ===")
print(df.head(), "\n")

```

```

# -----
# 2. Preprocessing, Dummy Formulation, and Variance Scaling
# -----

X = df.drop(columns=['satisfaction'])
y = df['satisfaction']

categorical_features = ['Customer_Type', 'Class']
numerical_features = ['Age', 'Flight_Distance', 'Inflight_Wifi_Quality', 'Seat_Comfort', 'On_time

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(drop='first'), categorical_features)
    ])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratif

X_train_p = preprocessor.fit_transform(X_train)
X_test_p = preprocessor.transform(X_test)

encoded_cat_features = preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_
all_features = list(numerical_features) + list(encoded_cat_features)

```

```

# -----
# 3. Binomial Optimization Execution
# -----

model = LogisticRegression(solver='lbfgs', penalty='l2', random_state=42, max_iter=1000)
model.fit(X_train_p, y_train)

y_pred = model.predict(X_test_p)
y_prob = model.predict_proba(X_test_p)[: , 1]

```

```

# -----
# 4. Operational Classification Reports & Plots
# -----

print("=== EVALUATION METRICS ===")
print(f"Accuracy:  {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred):.4f}\n")

# Confusion Matrix Generation
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 3.5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Unsatisfied', 'Satisfied'],
            yticklabels=['Unsatisfied', 'Satisfied'])
plt.title('Performance Confusion Matrix')
plt.ylabel('Ground Truth')
plt.xlabel('Prediction')
plt.tight layout()

```

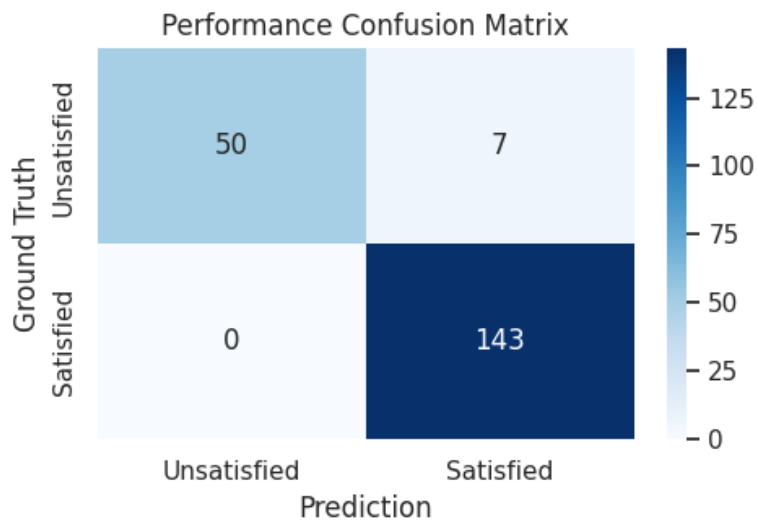
```
plt.show()
```

```
=== EVALUATION METRICS ===
```

```
Accuracy: 0.9650
```

```
Precision: 0.9533
```

```
Recall: 1.0000
```



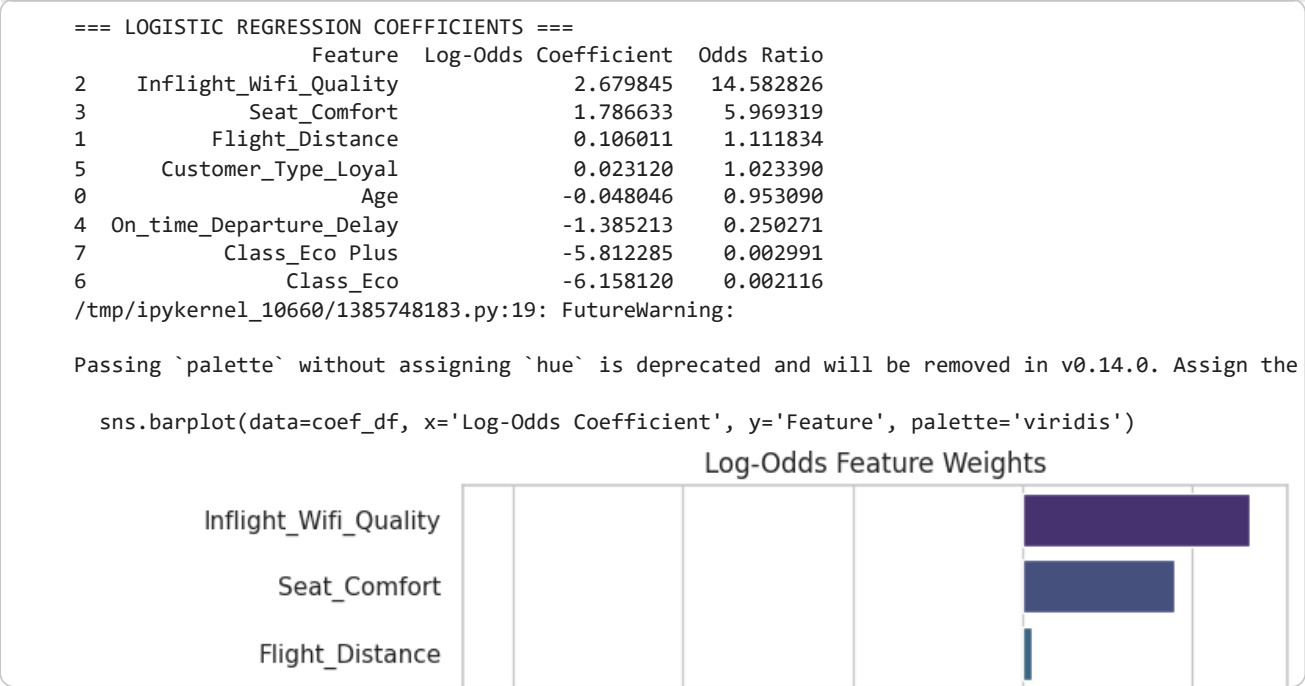
```
# -----
# 5. Strategic Coefficient Evaluation
# -----

coefficients = model.coef_[0]
odds_ratios = np.exp(coefficients)

coef_df = pd.DataFrame({
    'Feature': all_features,
    'Log-Odds Coefficient': coefficients,
    'Odds Ratio': odds_ratios
}).sort_values(by='Odds Ratio', ascending=False)

print("=== LOGISTIC REGRESSION COEFFICIENTS ===")
print(coef_df)

# Feature Importance Plot
plt.figure(figsize=(8, 4.5))
sns.barplot(data=coef_df, x='Log-Odds Coefficient', y='Feature', palette='viridis')
plt.title('Log-Odds Feature Weights')
plt.tight_layout()
plt.show()
```



7. Formulate Business Recommendations

Based on the statistical outputs calculated above, we can draw the following data-backed business insights:

- Prioritize High Odds-Ratio Elements:** Identify features with the highest positive coefficients (e.g., `Inflight_Wifi_Quality` or `Seat_Comfort`). A 1-unit increase in these metrics multiplies customer satisfaction odds by the corresponding Odds Ratio.
- Mitigate Friction Points:** Features with negative coefficients (like `On_time_Departure_Delay`) drag down satisfaction. The airline can set critical threshold alerts (e.g., when delays cross a time limit that severely drops satisfaction probabilities).
- Class-Specific Retention Strategies:** The dummy variable coefficients (e.g., `Class_Eco`) reveal exactly how much lower satisfaction odds are compared to the baseline (`Class_Business`), justifying targeted perk updates for economy travelers.